



Toward IoT device fingerprinting from proprietary protocol traffic via key-blocks aware approach

Yafei Sang^{a,b,*}, Jisong Yang^{a,b}, Yongzheng Zhang^c, Shuhao Li^{a,b}, Peng Chang^{a,b}

^a Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

^b School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

^c China Assets Cybersecurity Technology, Beijing, China

ARTICLE INFO

Article history:

Received 7 May 2022

Revised 3 December 2022

Accepted 19 February 2023

Available online 21 March 2023

Keywords:

IoT device fingerprint

IoT device identification

Proprietary protocol fingerprint

Deep clustering

Sequence alignment

ABSTRACT

IoT Device fingerprinting of network traffic is valuable for many management and security solutions as it provides insights into the devices active on a network. Unfortunately, existing techniques focus on the public (de facto) standard protocol and suffer from low efficiency and high data packet dependence at flow-level granularity. Hence, accurately identifying the types of proprietary-protocol-oriented IoT devices is still a challenge that cannot be ignored. As a solution, we propose DeepFinger, a full-automated, byte-level payload fingerprint generation approach to fill this gap. DeepFinger aims to eliminate the manual intervention required to extract available payload-driven fingerprints, in the absence of priori protocol specification information. The key sight of DeepFinger is that it utilizes **deep** clustering to automatically cluster similar payloads and infer key-blocks as **fingerprints**. Through extensive evaluation, we demonstrate that DeepFinger achieves the average TPR with 98.81%, the average FTF with 98.74%, and the average FPR with 0.07% on the dataset containing multiple proprietary protocols. In addition, on three datasets containing public protocols such as MQTT, Modbus, HTTP, and XMPP, DeepFinger also achieved excellent performance by virtue of its extensibility. These results suggest that DeepFinger can be a promising tool for automatizing the payload fingerprint extraction process, based on proprietary protocol assumption.

© 2023 Published by Elsevier Ltd.

1. Introduction

1.1. Background

With the widespread use of IoT devices in daily life and industrial production, the “Internet of Things” era is gradually coming. According to a research report from intelligent data systems (Riad et al., 2020), there are 64 billion IoT devices connected to the internet in 2025. Since different IoT devices have dissimilar Quality of Service (QoS) requirements, and the communication security between devices needs to be guaranteed at distinct limits, IoT device identification is important for network security and management strategies.

The behavior sources of IoT device identification are diverse including Radio Frequency (RF) (Jafari et al., 2018; Riyaz et al., 2018), Clock Skew (Lanze et al., 2012; Oser et al., 2018), and Network Traffic (O'Connor et al., 2019; Sivanathan et al., 2020; Thangavelu et al., 2018). Although the methods based on RF and Clock Skew

are effective, they are limited in application scenarios, such as the need to use complex instruments at close range to obtain behavior characteristics. Considering versatility and easy collection properties of traffic, it is necessary to realize IoT device identification based on the communication traffic between device and server. According to the implemented protocol, IoT device traffic can be divided into two types: **public protocol** (such as CoAP, MQTT, AMQP, XMPP) traffic and **proprietary protocol** traffic. The term **proprietary protocol** (also known as closed protocol or unknown protocol) refers to the IoT communication protocol that is developed by companies, projects, and individuals on specific demands such as simple functions and fast communication, and have a lack of public documents.

The research by cloud-based security provider Zscaler found that 83% of IoT-based transactions are occurring over plain-text channels (Zscaler, 2020), including most communications based on proprietary protocols. As shown in Fig. 1, identifying the proprietary-protocol-oriented IoT device mainly uses the labor-intensive manual method to model proprietary protocol traffic to obtain device fingerprints, but this faces some challenges.

* Corresponding author at: Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China.

E-mail addresses: sangyafei@iie.ac.cn (Y. Sang), yangjisong@iie.ac.cn (J. Yang).

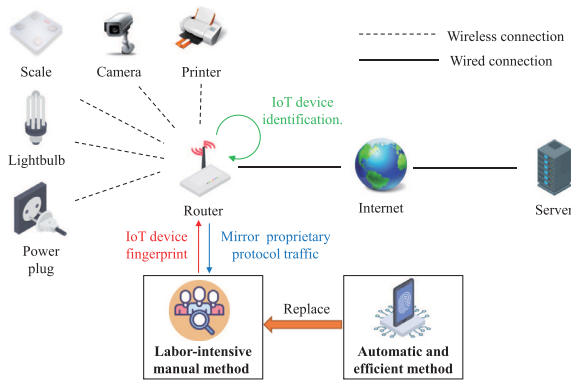


Fig. 1. Illustration of IoT device fingerprinting.

- Although active probing is widely used by security companies and researchers, most of the open services of IoT devices do not have enough information for device type identification.
- A majority of non-standard proprietary protocols have a lack of publicly available documentations, although there are standard RFCs for the standard public protocols.
- The labor-intensive manual device fingerprint extraction process has to be repeated from time to time so as to maintain the latest signature repository.
- Some protocol reverse engineering can generate proprietary protocol fingerprints but suffer from low efficiency. Since the number of proprietary protocols involved in IoT device traffic is huge, protocol reverse engineering is not a suitable choice.

Thus, how to efficiently and automatically generate fingerprints of proprietary-protocol-oriented IoT devices remains a challenging issue.

1.2. Limitation of prior art

It has been previously observed that plentiful high-precision techniques have appeared in the identification of IoT devices (Aneja et al., 2018; Apthorpe et al., 2017; Lopez-Martin et al., 2017; Maiti et al., 2017; Marchal et al., 2019; Meidan et al., 2017). The existing methods can roughly be divided into two categories: *feature engineering based* and *automatic learning based*. Compared to the former exploits the complex and handcrafted(domain-expert drive) statistical characteristics and field characteristics of packet payload for fingerprint generation, the primitive and simple packet attributes(such as payload length, message type) in a direct manner to use in the latter. However, these methods mainly focus on public protocols and have the following inherent limitations in the real IoT environment.

Complex feature extraction and narrow applicability The *feature engineering based* methods need to manually extract the statistical characteristics and field characteristics of the traffic, such as flow volume, flow duration, average flow rate, device sleep time, server port numbers, cipher suites. For statistical characteristics, the selection process is subjective and complex, which results in the generated device fingerprints being statistically sensitive and lacking in robustness. For field characteristics, due to the unclear format and key fields of the proprietary protocol, the field characteristics of the packet payload cannot be directly parsed and extracted, thus it is arduous to model and classify the traffic.

High data dependence and low efficiency The early-identification is an important goal in the task of IoT device identification, which means that only a small number of packets at the beginning of the flow are used for device identification. However, whether it is the *feature engineering based methods* or *automatic engineering*

based methods, all or most of the packets in a flow need to be used. This high dependence on data makes it take a long time to obtain multiple packets, which makes early-identification fundamentally difficult to achieve. Furthermore, due to the use of the neural network model, a large number of matrix multiplications are required to calculate in the identification phase by *automatic learning based methods*, which greatly reduces the identification efficiency. Hence, it is a colossal challenge to process the high-speed proprietary protocol traffic in the real IoT environment with the existing methods.

To sum up, we find that the above methods lack the ability to effectively construct fingerprints of proprietary-protocol-oriented IoT devices, and the high dependence on data makes early-identification difficult. In addition, affected by the lack of protocol format documentations, the methods that require prior knowledge of the protocol format cannot be directly migrated to generating proprietary protocol device fingerprints. Thus, the primary goal of this study is to address the limitations of existing methods for generating fingerprints for proprietary-protocol-oriented IoT devices and provide more accurate fingerprint extraction in an unsupervised way.

1.3. Key contributions

In this research paper, we propose a proprietary-protocol-oriented IoT device fingerprint generation method based on payload key-blocks awareness. In summary, our main contribution in this paper are includes the following::

- We propose DeepFinger, an innovative automatic, byte-level, and efficient fingerprint generation approach for proprietary-protocol-oriented IoT devices.
- We construct a deep clustering model to automatically extract the *key-blocks* in the packet payload and utilize a sequence alignment algorithm to generate IoT device fingerprints from the packet payload clusters.
- DeepFinger possesses low dependence on traffic data and realizes early-identification. Considering that the IoT proprietary protocol has the characteristics of simple rules and fewer formats, our model only utilizes the first packet payload in the flow.
- DeepFinger converts fingerprints into regular expressions for string matching to identify IoT devices, which enjoy the characteristics of strong availability and easy deployment and controls the average time of identification at the nanosecond level.
- DeepFinger possesses an excellent IoT device identification ability. For the datasets containing 17 IoT devices and multiple proprietary protocols, DeepFinger has an average TPR of 98.81%, an average FTF of 98.74%, and an average FPR of 0.07%. Furthermore, the effectiveness of our proposed method is validated on Edge-IIoTset, IoT Sentinel, and MedBioT datasets with known protocol formats, including 29 devices and several public protocols such as MQTT, Modbus, HTTP, XMPP, etc.
- We release DeepFinger as an open-source tool in the community, which can be obtained from the repository¹ and allow future extensions.

The rest of this paper is organized as follows. Section 2 introduces the IoT protocol and defines the problem. In Section 3, we introduce the research motivation and research method of this paper. In Section 4, we overview the DeepFinger designed and elaborate on the technical details. In Section 5, we conduct the experiments of DeepFinger and present diverse experimental results. In Section 6, we discuss the performance and future improvements of DeepFinger. In Section 7, we review related work. Finally, we conclude the paper in Section 8.

¹ <https://github.com/paper2project/DeepFinger>

2. Preliminaries

2.1. Understanding IoT protocols

The IoT protocol is usually used for communication between IoT devices and the server or control end. In general, a single IoT device possesses the simple functions and requires high communication efficiency, which reduces the complexity of the protocol format design. In addition, IoT devices once turned on, have three possible states, i.e., initialization, idle, active. Different states have different connection modes, such as the need to send heartbeat traffic between the device and the server in the idle state. These scenarios lead to many unique characteristics of the IoT protocol. The LIFX protocol² is a binary messaging protocol that is used to communicate with LIFX devices, sending them instructions or querying them for state. We take the LIFX protocol as a typical example to discuss the details of the following three characteristics.

Flat format The structure of most IoT protocols composes a fixed-length header followed by an optional payload. Each LIFX protocol message has two components concatenated together: a header(36 bytes) and a payload. The header contains metadata common to all messages and describes the type of payload to follow; the payload provides the data for the specific action being requested

No delimiters Most IoT protocols are simply designed and lack delimiters. The LIFX protocol is a binary messaging protocol with no delimiters.

Short key fields Although the fields of traditional network protocols have a relatively big length, there are many short key fields in IoT protocol. For example, both the *addressable* field and the *tagged* field in LIFX only occupy one bit.

2.2. Problem statement

On the premise of traffic based, IoT device identification is defined as the act of ascertaining which IoT device is contributing to the network traffic by using a fingerprint, thereby further determining the type of device corresponding to the traffic. Fingerprints in this paper refer to the distinguishable positions and values of specific byte blocks in the payload, as well as their own distinguishable relations. Regardless of the TLV(Tag-Length-Value) type protocol or the Binary type protocol, there are such distinguishable byte blocks, which we name *key blocks*. Correspondingly, the indistinguishable field is called *stochastic-blocks*. Figure 2 shows the *key-blocks* are the instances of the protocol key field. The green part represents the *key-blocks*, and the instance types of the *key-blocks* are relatively few. The orange part represents the *stochastic-blocks*, and there are relatively many types of *stochastic-blocks* instances. Figure 2 also shows multiple hexadecimal packets of the two protocols SensorID³ and MELSEC.⁴ It can be found that the packet fields corresponding to the *key-blocks* are relatively fixed, while the packet fields corresponding to the *stochastic-blocks* are relatively random.

To formally describe the problem we want to solve, we make the following problem definition. We define bytes table $T = \{0, 1, \dots, 255\}$. A block is a continuous bytes sequence and we define block set $B = \{b | b = t_1 t_2 \dots t_n \wedge t_i \in T\}$. The payload $p_i = f_1 f_2 \dots f_m$ is spliced by protocol fields. These fields will be mapped to specific blocks in the instance, such as $f_1 \rightarrow \{b_1, b_2\}$, $f_2 \rightarrow \{b_1, b_2, \dots, b_n\}$. If the blocks mapped by a field are relatively fixed, these blocks are defined as *key-blocks*, otherwise are defined as

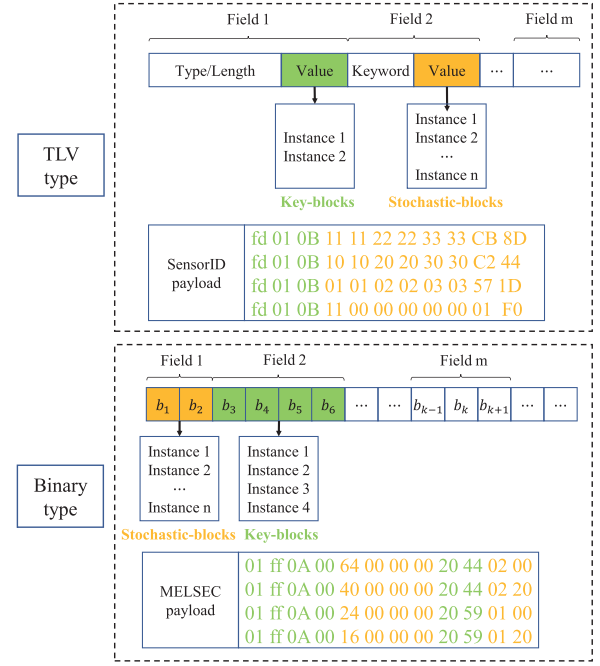


Fig. 2. Illustration of protocol fields, blocks and instances.

stochastic-blocks. All *key-blocks* constitute a set B_{key} . Given the payload set $S = \{p_1, p_2, \dots, p_n\}$, our goal is to find the objective function $h(S) \rightarrow B_{key}$ and $g(S) \rightarrow F$, where B_{key} represents the set of key blocks, F represents the set of fingerprints.

3. Motivation and our research

Our research presents DeepFinger, an approach that automatically infers IoT device fingerprints and effectively identifies the IoT device type corresponding to each flow from the mixed traffic. DeepFinger infers fingerprint from application layer packet payload that uses proprietary protocols. Intuitively, each device fingerprint is composed of *key-blocks* in the packet payload and expressed as a regular expression for byte-level sequence matching.

Our work is based on the following simple but fundamental intuition. The protocol consists of a series of fields, whether it is a public protocol or a proprietary protocol, the values of these fields are either relatively fixed or relatively random. We perform statistical calculation on the packet payload of multiple flows from the same device, and count the type and number of bytes appearing in each location. Further, we calculate the entropy at the current location and try to show the randomness of the field by the distribution of entropy over different byte locations. Specifically, all bytes appearing in a certain are regarded as a sequence containing n different bytes and each byte is in the set $\{e_1, e_2, \dots, e_n\}$. The entropy of this certain location is defined as $-\sum_{i=1}^n \frac{m_i}{m} \log_2 \frac{m_i}{m}$, where m_i is the number of occurrences of byte e_i , and m is the sequence length at this certain location.

Figure 3 shows the distribution of entropy at different byte locations for packet payloads from several IoT devices. Some byte locations have the very low entropy, which means that the bytes appearing at this position for all payloads are relatively fixed and are expected potential *key-blocks*. Conversely, the locations with particularly high entropy indicate strong randomness and are not discriminative, and it has a high probability of being the *stochastic-blocks*.

It is worth noting that for some locations at the median entropy value, they may be variable fields with a small number of byte value types, such as the field representing the type of the reg-

² LIFX, <https://github.com/paper2project/DeepFinger>

³ SensorID, <http://www.sensorid.it/>

⁴ MELSEC, https://www.mitsubishielectric.com/fa/cn_zh/products/cnt/plc/index.html

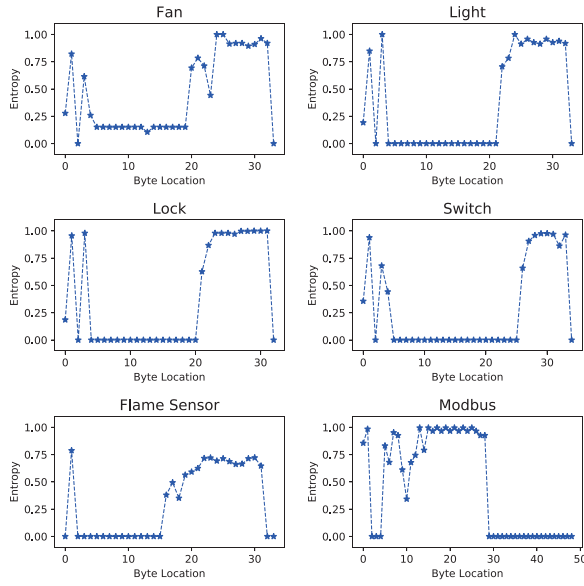


Fig. 3. Payload byte entropy distribution for different devices.

ister in the MELSEC protocol, or the field representing action commands for smart home devices. If these relatively variable fields can be extracted separately, they can also be used as discriminative fingerprints. Therefore, we try to cluster the same original packet, expecting to cluster the packets containing the same *key-blocks* together, such as clustering the packets of the MELSEC protocol into four clusters, each of which contains the same register type.

The goal of DeepFinger is to extract all *key-blocks* to form a payload fingerprint. To implement the intuition, we face three major technical challenges in designing our DeepFinger approach:

- How to aggregate the packet payload containing the same *key-block* without a priori protocol format information?
- How to accurately perceive the values of the *key-blocks* from the payload clusters, and generate a *key-blocks* set?
- How to generate and refine the device payload fingerprint based on the *key-blocks* set?

To address the first challenge, we proposed a deep clustering method that integrates deep learning models and clustering algorithms, which can accurately and meticulously obtain clusters. Compared with other works that require the manual definition of complex indicators for clustering, our method is fully automatic and treats the same *key-blocks* in different locations as different *key-blocks*. It's worth noting that the packet payload in each cluster has the same *key-block*, but the *key-blocks* of different clusters may be the same.

To address the second challenge, considering that most proprietary protocols consist of short fields with no delimiters, we use the *n*-gram method to realize *key-block* perception in each cluster. Although these *key-blocks* can be used directly as device fingerprints, there are still two limitations. i) The *key-blocks* perceived by the *n*-gram are independent and do not represent the protocol format relationship, which leads to the low recall rate. ii) Since different clusters may contain the same *key-block*, it will bring too many redundant fingerprints and make the precision rate low. Therefore, we merge the clusters that contain the same *key-block* to make the *key-blocks* of different clusters unique.

To address the third challenge, we use a biological gene sequence alignment algorithm to generate fingerprints containing *key-blocks* from each merged cluster. The packet payloads in each merged cluster contain the same *key-blocks*, but these *key-blocks* may appear at different locations in the payload byte sequence.

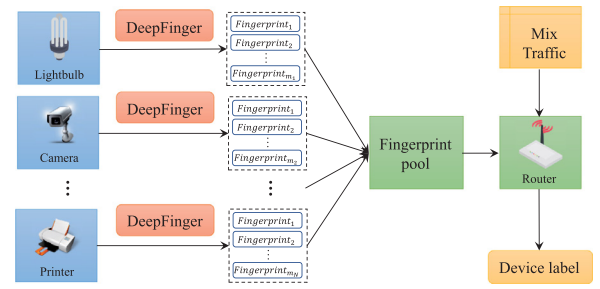


Fig. 4. Application scenario of DeepFinger.

The sequence alignment algorithm based on dynamic programming can align and output these *key-blocks* at different locations.

Figure 4 shows the application scenario of DeepFinger in IoT device identification. DeepFinger generates a series of fingerprints belonging to the current device for the traffic of each device, such as Lightbulb, Camera, and Printer. Then, the fingerprints of all devices are collected and merged into a fingerprint pool which is deployed to the middleboxes (such as routers, firewalls, IDS, IPS) at the gateway to identify the IoT devices connected.

4. DeepFinger

4.1. Overview

DeepFinger contains five modules and four intermediate data forms. To better understand how DeepFinger generates the device fingerprint from the traffic of a single device, Fig. 5 represents the main modules of DeepFinger and the changes in the data form between the modules.

First, each flow in the raw traffic set of a single IoT device passes through the Data Preprocessing module to get the First Packet Payload, the intercepted bytes of the first packet of the current flow, including forward bytes and reverse bytes. Different First Packet Payload contains their own *key-blocks*, which are distinguished by symbols of different colors and shapes in the figure. Second, all First Packet Payloads go through the Deep Clustering module together to obtain *k* Same Key-Block Clusters. The First Packet Payloads in each Same Key-Block Cluster contain the same *key-blocks*, and the *key-blocks* corresponding to different Same Key-Block Clusters may also be the same, such as $SKBC_1$ and $SKBC_{k-1}$ in the Fig. 5. Third, the Keyword Perception module extracts the *key-blocks* of each cluster and merges clusters containing the same *key-blocks* to obtain *m* Key-Block Sets, where *m* is not greater than *k*. Fourth, each Key-Block Set can generate IoT Device Fingerprints through the Fingerprint Generation module. Finally, the Device Identification module obtains the IoT device label based on the fingerprints.

Next, we introduce the technical details of the five modules in DeepFinger. The notations used in this paper are shown in Table 1.

4.2. Data Preprocessing

We denote the set of packet payloads from the same IoT device as $\mathcal{S}$. Each payload in $\mathcal{S}$ contains the unique format fields of the device, and we find out the specific embodiment *key-blocks* of these unique format fields as the fingerprint of the device. For each packet payload $p_i \in \mathcal{S}$, $p_i = \{b_{i1}, b_{i2}, \dots, b_{im}\}$, where b_{ij} is the *j*-th byte in the payload p_i . Considering that the length of the packet payload is not uniform, and the distribution of *key-blocks* is close to the front and back ends, we intercept *L* bytes from the forward and reverse directions respectively, denoted as forward bytes $p_{if} = \{f_{i1}, f_{i2}, \dots, f_{iL}\}$ and reverse bytes $p_{ir} = \{r_{i1}, r_{i2}, \dots, r_{iL}\}$. We

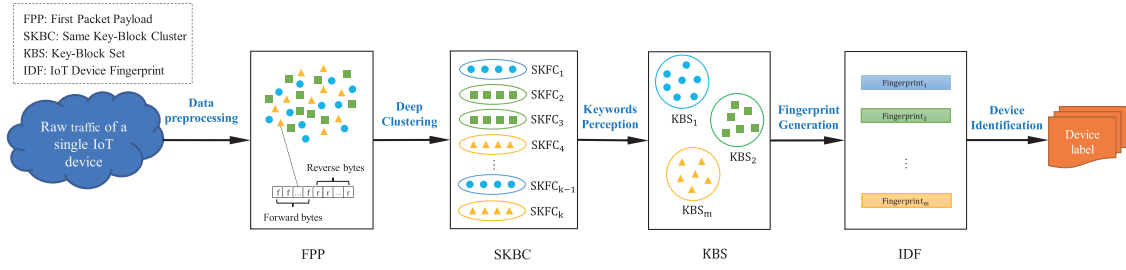


Fig. 5. The workflow of DeepFinger.

Table 1
List of notations in this paper.

Notations	Meaning
S	Packet payloads set
p_i	A packet payload in S
b_{ij}	A byte in packet payload p_i
f_{ij}	Forward j -th byte in p_i
r_{ij}	Reverse j -th byte in p_i
L	Input bytes length
p_{if}	Forward input bytes in p_i
p_{ir}	Reverse input byte in p_i
$\mathcal{F}$	CNN output vector
$K(\mathcal{F})$	Clustering result
C_j	The j -th cluster in $K(\mathcal{F})$
z_j	Clustering label
y_n	Pseudo-label
K	Keywords set

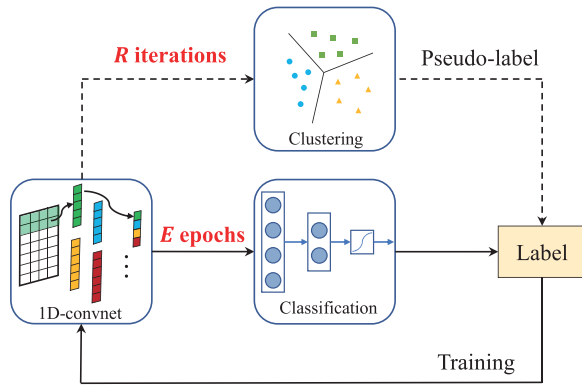


Fig. 6. The architecture of deep clustering.

treat L here as a key parameter, which we will discuss in the following evaluation experiments. The p_{if} and p_{ir} are the data preprocessing results of p_i and are used as the input of subsequent modules.

4.3. Deep clustering

As a general rule, an IoT device possesses multiple fingerprints, which prompts us to divide S into multiple clusters. We aim to make the *key-blocks* of the payloads in each cluster as identical as possible, and we plump for the clustering algorithm to accomplish this assignment. Nevertheless, a single clustering algorithm is not appropriate for this assignment in that the payload has no effective features for clustering. Further operations or processing are required to obtain clustering features, and it is laborious to evaluate the effectiveness of features. In this article, we propose a deep clustering method shown in Fig. 6, including three steps.

Step1. Feature generation We extract the clustering features of each data packet payload automatically and straightforward by the neural network. The p_f and p_r are passed through a one-

dimensional convolutional neural network (CNN) and a fully connected layer successively. Then we concatenate the output vectors as the clustering feature $\mathcal{F}$:

$$\mathcal{F} = \text{concat}(fc(\text{cnn}_{\theta}(p_f), fc(\text{cnn}_{\theta}(p_r)))) \quad (1)$$

where cnn represents the convolutional neural network, θ is the corresponding parameter set of the convolutional neural network, and fc represents the fully connected layer. We configure three convolution kernels for the one-dimensional convolution, taking into account efficiency and effectiveness. The 1-D CNN consists of an embedding layer with 128 dimensions, a convolution layer, and a fully connected layers. There are three convolution cores in the convolution layer, which are set as 2,3,4, and the out channels is set as 128, the stride is set as 1. The activation function selects the ReLU function and pooling layers uses 1-D max pooling.

Step2. Feature clustering. In the absence of clustering features particularity, we select the standard K-Means to cluster the features $\mathcal{F}$ of the same device. The clustering result is denoted as $K(\mathcal{F})$, which contains N payloads, k clusters. Each cluster $C_j \in K(\mathcal{F})$ contains several packet payloads that possess the same *key-blocks*. We numbered each cluster with the clustering result $z_j \in \{1, 2, 3, \dots, k\}$.

$$K(\mathcal{F}) = \begin{cases} z_i = \arg \min_{1 \leq j \leq k} \|x_i - \mu_j\| \\ \mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i \end{cases} \quad (2)$$

For the packet payloads in the cluster C_j , we apply j as their pseudo-label y_n in the current iteration round.

$$y_n = j, \quad j \in [1, k], \quad n \in [1, N] \\ \text{s.t. } p_n \in C_j \wedge C_j \in K(\mathcal{F}) \quad (3)$$

Step3. Model training After obtaining the pseudo-labels of all payloads in S , we implement the training of our convolutional neural network by pseudo-labeled data. The essence is to find the optimal parameter θ by optimizing the following problem:

$$\min_{W, \theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(g_{W, \theta}(p_n), y_n) \quad (4)$$

where $\mathcal{L}$ is the loss function, $g_{W, \theta}$ is the integrated bidirectional CNN classification network, and y_n is the pseudo-label corresponding to the payload p_n . Normally, we take advantage of backpropagation and gradient descent to optimize (4).

Overall, as shown in Algorithm 1, the Deep Clustering module alternately generates clustering features, payload clusters, and pseudo-labels by iterating the (1)–(4) in multiple rounds. This module updates the parameters of the neural network regularly and attains the results on the clustering problem of the payload *key-blocks*.

4.4. Keywords perception

The purpose of this module is to perceive the *key-blocks* of the payload format from each cluster. The n -gram method considers consecutive n bytes as a tuple, and does not consider the position

Algorithm 1 Deep clustering.

Input: Payload Set $\mathcal{S}$, Iteration Round R , Training Epoch E , CNN Model $g_{W,\theta}$, Fully Connected Layer fc
Output: Clusters C

```

1: Pseudo-label  $pl = \text{RANDOM}$ 
2: for round  $r = 1; r \leq R; r++$  do
3:   for epoch  $e = 1; e \leq E; e++$  do
4:      $g_{W,\theta} \leftarrow \text{UPDATE}(g_{W,\theta}, \mathcal{S}, pl)$ 
5:   end for
6:   Feature  $\mathcal{F} \leftarrow$  output of  $fc$ 
7:   Cluster-label  $cl$ , Clusters  $C \leftarrow KMeans(\mathcal{F})$ 
8:    $pl \leftarrow cl$ 
9: end for
10: return  $C$ 

```

information of the bytes. However, the packet payloads *key-blocks* in each cluster have great similarities after deep clustering, and the length of the *stochastic-block* is subject to certain restrictions in the single cluster. Fully exploiting location-based information to perceive *key-blocks* is an explicit and effective method.

Algorithm 2 demonstrates the core operation steps of our Key-

Algorithm 2 Keywords perception.

Input: Clusters C , PayloadLength L , Threshold T
Output: Key Block Set K

```

1:  $K \leftarrow []$ 
2: for cluster  $C_i \in C$  do
3:   ClusterKeywords  $K_i \leftarrow []$ 
4:    $S_i \leftarrow \text{ngram}(C_i)$ 
5:   for location  $loc \in \text{range}(L)$  do
6:      $LC_{i,loc}, LK_{i,loc} \leftarrow \text{Count}[S_i[loc]]$ 
7:     if  $LC_{i,loc} \leq T$  then
8:        $\text{ADD}(K_i, LK_{i,loc})$ 
9:     else
10:       $\text{ADD}(K_i, \text{None})$ 
11:     end if
12:   end for
13:    $\text{ADD}(K, K_i)$ 
14: end for
15: return  $K$ 

```

words perception module. For each cluster C_i , we generate the n -gram sequence S_i of all payloads and count the number of n -gram types in the same position. If the number of n -gram types $LC_{i,loc}$ is overly large, it presumably implies that the n -gram byte $LK_{i,loc}$ at the current position is a random byte, which means that the current position is not a key position, so add *None* to the cluster keywords set K_i . Otherwise, we regard the $LK_{i,loc}$ as a keyword byte of *key-blocks* and add it to K_i . In consideration of the accuracy of clustering, for the threshold T , we initialize $T = 2$ or $T = 3$ and subsequently modify the Initial value by the IoT device identification result. Eventually, the module outputs the final Key Block Set K which including the *key-blocks* of each cluster.

4.5. Fingerprint generation

The purpose of this module is to merge multiple clusters with the same Key Block Set to generate fingerprints that can be directly used for device identification. In the previous module, we perceive the *key-blocks* of each location for each cluster. If these *key-blocks* are converted into regular expressions, we essentially generate the fingerprints that match all payloads in this cluster. However, we will not apply these regular expressions as final fingerprints for device identification. At least one fingerprint is generated for each

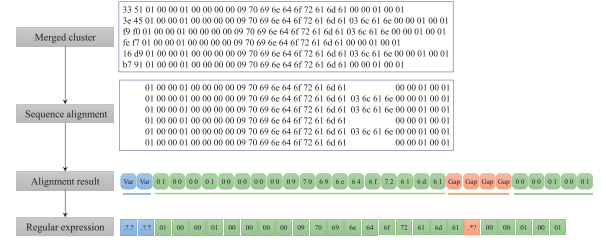


Fig. 7. An example of fingerprint generation.

cluster, and the number of clusters is far greater than the number of real payload fingerprints, which increases the redundancy of device identification. Besides, the raw fingerprints are generated based on location information and do not contain *stochastic-block*, which increases the complexity of device identification.

To address the aforementioned issues, this module proposes a method of merging clusters and generate fingerprints. Initially, we ignore the location information and find clusters with the same *key-blocks*, and merge these clusters into a large cluster. For each new cluster, all data packet payloads contain identical *key-blocks* even if the locations are not fixed. Then we utilize the sequence alignment algorithm to generate a universal matching sequence from merged payloads as the final device fingerprint. In DeepFinger, we take advantage of the well-known Needleman-Wunsch algorithm for sequence alignment (Needleman and Wunsch, 1970) (Beddoe, 2004), which is based on dynamic programming to find sequence matches.

Figure 7 shows the whole process of fingerprint generation, including merged payload clusters, sequence alignment results and the examples of producing regular expressions. *Var* indicates that the current position is a fixed-length *stochastic-block*, and *Gap* indicates that the current position is filled with a blank due to the position offset of the *key-blocks*. We convert each *Var* to $(.?.?)$, one or more consecutive *Gap* to $(.*?)$, and the other bytes remain unchanged, thus obtaining a regular expression of a fingerprint.

4.6. Device identification

This module implements IoT device identification based on the obtained fingerprints, which exist in the form of regular expressions. For a new flow, we extract the payload of its first packet, then use regular matching to check its corresponding fingerprint, and get the predicted type label of the IoT device. It is worth noting that a packet payload should pass the regular matching check of all fingerprints, which is a large amount of engineering. However, DeepFinger is still efficient than the deep learning model, since the identification phase DeepFinger omitted matrix operations.

5. Experimental evaluation

In this section, we present a series of experiments to evaluate the effectiveness and efficiency of DeepFinger. First, we introduce the dataset used for the evaluation, showing the details of the evaluation metrics and the initial settings of our method. Then, we perform a sensitivity analysis on the hyperparameters in the method and compare our method with related methods. In addition to this, we also evaluate the time complexity of our DeepFinger. Finally, we discuss the extensibility of our method in public protocol traffic.

5.1. DataSet

UNSW2016 UNSW2016 (Sivanathan et al., 2018) is captured from a real-world IoT network environment to train and test DeepFinger.

Table 2
Summary of dataset in evaluation.

ID	IoT device	MAC address	Flows	Packets
0	Amazon Echo	44:65:0d:56:cc:d3	55.3 K	1.3 M
1	Belkin Motion Sensor	ec:1a:59:83:28:11	41.5 K	1.4 M
2	Belkin Switch	ec:1a:59:79:f4:89	36.0 K	1.1 M
3	Dropcam	30:8c:fb:2f:e4:b2	152.1 K	4.2 M
4	HP Printer	70:5a:0f:e4:9b:c0	4.6 K	248.3 K
5	iHome PowerPlug	74:c6:3b:29:d7:1d	2.6 K	65.4 K
6	LiFX Bulb	d0:73:d5:01:83:08	8.6 K	172.7 K
7	Netatmo Camera	70:ee:50:18:34:43	16.7 K	673.7 K
8	Netatmo Weather station	70:ee:50:03:b8:ac	6.4 K	276.8 K
9	Pixstart photo frame	e0:76:d0:33:bb:85	7.2 K	68.7 K
10	Samsung Smart Cam	00:16:6c:ab:6b:88	26.1 K	1.2 M
11	Smart Things	d0:52:a8:00:67:5e	8.2 K	571.0 K
12	TP-Link Camera	f4:f2:6d:93:51:f1	11.7 K	319.0 K
13	TP-Link Plug	50:c7:bf:00:56:39	5.5 K	41.4 K
14	Tribby Speaker	18:b7:9e:02:20:44	5.9 K	200.6 K
15	Withings Scale	00:24:e4:1b:6f:96	1.3 K	5.6 K
16	Withings sleep sensor	00:24:e4:20:28:c6	6.4 K	387.8 K

The dataset collects over a period of 26 weeks of IoT device traffic, including motion sensor, camera, speaker, etc, and applies the MAC address as the label of each IoT device. It is worth noting that this dataset is not entirely private protocol traffic. We use tshark to filter out the TLS protocol and public protocols, and treat the remaining traffic as proprietary protocol traffic. Although this strategy still does not guarantee that the traffic obtained applies the proprietary protocols, it does not affect the verification and evaluation of our method as we do not use any prior knowledge of the protocol format.

Edge-IIoTset Edge-IIoTset (Ferrag et al., 2022) is a new comprehensive realistic cyber security dataset of IoT and IIoT applications. All the traffic are generated from various IoT devices such as Low-cost digital sensors for sensing temperature and humidity, Ultrasonic sensor, Water level detection sensor, pH Sensor Meter, Soil Moisture sensor, Heart Rate Sensor, Flame Sensor. All devices use IoT standard public protocols, most of which use MQTT protocol, and a few use Modbus protocol.

IoT sentinel IoT Sentinel (Miettinen et al., 2017) is derived from a prototype system developed in an IoT lab environment that simulates the IoT device setup process in a typical home and small office environment. The dataset represents the traffic emitted during the setup of 31 smart home IoT devices of 27 different types, including sensors, switches, cameras, sockets, light bulbs, and more. The protocols involved in these devices include HTTP protocol, SSDP protocol, TFTP protocol, TLS protocol, etc.

MedBlot MedBlot (Guerra-Manzanares et al., 2020) is derived from a simulated system deploying three well-known botnet malware Mirai, BashLite and Torii, consisting of normal traffic and malicious traffic. This paper selects normal traffic as the evaluation data set, including four devices: Lock, Switch, Fan, and Light. Every device generates traffic based on the MQTT protocol. all the four datasets generated by excellent iot work and span multiple data acquisition times, multiple data orders of magnitude, and multiple iot protocols, including private and public protocols. In the following experiments, we use UNSW2016 to analyze various aspects of the performance of our method in extracting fingerprints from proprietary protocol traffic, and use the remaining three datasets to verify the generalization of our method on public protocols. The protocol structure of the public protocol is clear in the remaining three datasets, which can verify the validity and extensibility of our work.

Table 2 shows the composition details of the UNSW2016 dataset, consisting of 396+ thousand traffic flows referring to 17 IoT devices after packet recombination techniques. In view of data balance, each IoT device randomly selects the first packet payload

of 1000 flows. We divide the dataset into two groups, one group contains 80% payloads for training DeepFinger related parameters, and the other group contains 20% payloads for testing DeepFinger performance. We adopt 5-fold cross-validation to enhance the reliability of our experiments.

5.2. Setup

Evaluation metrics We evaluate the effectiveness of our method on each IoT device according to the True Positive Rate TPR_i and False Positive Rate FPR_i . Furthermore, we define the following three macro metrics for multiple classifications to quantitatively evaluate the effectiveness of DeepFinger. The parameter M is the total number of labels.

$$TPR_{AVE} = \frac{1}{M} \sum_{i=1}^M TPR_i \quad (5)$$

$$FPR_{AVE} = \frac{1}{M} \sum_{i=1}^M FPR_i \quad (6)$$

$$FTF_{AVE} = \frac{1}{M} \sum_{i=1}^M \frac{TPR_i}{1 + FPR_i} \quad (7)$$

Implementation To implement our experiments, we configured some initial values. For the CNN model in Deep Clustering module, the embedding vector is set to 128 and the sizes of three kernels are set as 2, 3 and 4. We take dropout (Hinton et al., 2012) with 0.4 ratio to avoid over-fitting, and the Adam optimizer (Kingma and Ba, 2014) with learning rate 0.00005 is used. We use Protocol Informatica⁵ as a tool for sequence alignment and set the weight parameters match = 2, mismatch = -2, gap = 1. The core part of DeepFinger is implemented with PyTorch. Our experiments were executed on a cluster machine where each node had 32 Octa-core Xeon processors running at 2.10 GHz with 128 GB RAM.

5.3. Sensitivity analysis

We take *input bytes length*, *feature size*, and *clusters number* as three key parameters. We discuss the selection of three key parameters in the training phase of DeepFinger and analyze relevant experimental results under the optimal parameters.

Input bytes length L is the number of bytes to be truncated front and back the payload, and two reasons prompt us to truncate the payload. First, the byte length of the packet payload varies widely, from tens to thousands. The parameters of the CNN model in the training phase would be disproportionate if we use the payload directly without any process. Although we can unify the dimensions of the input data through the padding, an excess of padding will cause asymmetries in the learned weights, leading to spatial bias (Alsallakh et al., 2020). Second, to reduce the complexity of the CNN model in the training phase, and make effective use of computer memory, it is necessary to limit the length of the input data. Based on the distribution of the payload length in the data set, this paper conducted experiments for $L \in \{4, 8, 16, 32, 64\}$ to study the effect of diverse L on device identification. Figure 8(a) presents the results of TPR and FPR on the test set under various values of L . We observe that the TPR of each IoT device has a marked increase as the value of L rises. Conversely, the FPR drops rapidly. However, from 32 to 64 bytes, all evaluation metrics no longer change significantly. Considering the balance of performance and training time, we choose 32 as the value of L in this paper.

⁵ <http://phreakocious.net/PI/>

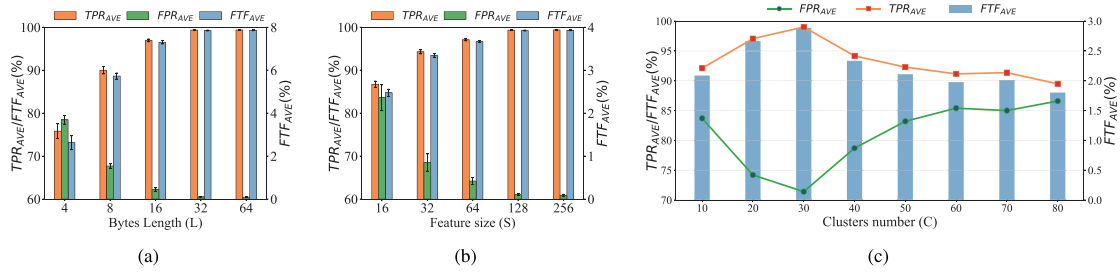


Fig. 8. Sensitivity analysis of fingerprint identification performance of DeepFinger. (a) different input byte lengths (b) different feature vector sizes (c) different clusters number.

Feature size S is the size of the convolution feature vector in the CNN model. We train DeepFinger with different feature size for $S \in \{16, 32, 64, 128, 256\}$ and show the results in Fig. 8(b). Obviously, as the feature size increase, the TPR rises while the FPR decreases. It is worthy to note that the improvement tends to be gentle with the exponential growth of the feature size. When the value of S is greater than 128, the change in the evaluation metrics is less than 0.1%. The CNN model will face a high risk of over-fitting even if dropout is applied when the feature size is very large, which reminds us that the feature size should be set reasonably according to the actual demand.

Clusters number Moreover, we evaluate the vital parameter of the clustering algorithm in *Deep Clustering* module. C is the maximum number of clusters in the K-Means algorithm. We anticipate that the payload in each cluster has the same *key-blocks*. When C matches the number of real fingerprints or the number of *key-blocks*, the clustering results will be extremely precise, thereby improving the effectiveness of the final device identification. Hence an appropriate value of C is an imperative consideration. For the data set in this paper, we do not have real fingerprints or *key-blocks* for each device, and cannot directly evaluate the clustering results of the *Deep Clustering* module, which prompts us to determine the appropriate C according to the final device identification result. This paper conducted experiments for $C \in \{10, 20, 30, 40, 50, 60, 70, 80\}$. Figure 8(c) shows the change of evaluation metrics on the test set with distinct values of C . As it turns out, the results are significant at the $C = 30$. The values of TPR performs best in the circumstances, and the values of FPR reaches minimum.

5.4. Comparison experiments

To demonstrate the competitiveness of DeepFinger, we conducted comparative experiments on three aspects, including baseline experiments, fingerprinting methods in other fields, and internal ablation experiments, including common sequence algorithm, Internet traffic fingerprint algorithm and APP traffic fingerprint, etc.

Some state-of-the-art and related models are summarized as comparison methods in the experiment as follows:

- BitCoding (Hubballi and Swarnkar, 2018) only takes the first small bits of the payload as a signature and translates the compressed signatures into a state transition machine. The number of bits is set as 20.
- LSTM is an improved recurrent neural network that can solve long-distance dependence problems. The word vector dimension is set as 32, the hidden size is set as 256, and the output dimension is set as 128. The improved variant of LSTM (biLSTM) was also used for comparison.
- CNN is a kind of feedforward neural network with convolution calculation and deep structure. The embedding size is set as 128, the out channels is set as 128, the stride is set as 1, and the sizes of three kernels are set as 2, 3 and 4.

- FoSM (Korczyński and Duda, 2014) builds the first-order Markov model and takes the flow with the maximum probability as the classification result.
- SOB (Shen et al., 2017) incorporates the attribute bigrams into the second-order homogeneous Markov chains. We take 100 as the number of bigram clustering.
- MaMPF (Liu et al., 2018) uses the output probabilities of Markov models as features to classify traffic with the random forest classifier. We take 40 as the number of trees.
- FS-Net (Liu et al., 2019) is an end-to-end classification model that adopts a multi-layer encoder-decoder structure, and it can mine the potential sequential characteristics of flows deeply.

In addition, we design some variants of our model which take advantage of other clustering algorithms. DeepFinger-birch adopts the BIRCH (Balanced Iterative Reducing and Clustering Using Hierarchies) algorithm, which uses the clustering feature tree to quickly cluster, and clustering can be performed only by scanning the data set in a single pass. Since we lack prior knowledge of the *key-blocks* in the data set, we will set the number of clusters to *None*. DeepFinger-dbscan adopts the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm, which assumes that the clusters can be determined by the tightness of the sample distribution, and hence closely connected samples are classified into one cluster. We utilize scikit-learn to help implement DBSCAN and set the parameters $eps = 0.3$, $min_samples = 30$.

The comparison results are shown in Table 3. For all methods, we adopt the 5-fold cross-validation experiment and demonstrate the average and standard deviation of each evaluation metrics. From Table 3, DeepFinger variants outperform all the other methods, and DeepFinger achieves the best performance. BitCoding has established a ternary signature for each device, and the training process is very simple. However, an IoT device possesses multiple *key-blocks* in most cases. Consequently, it is difficult for a single signature to correctly classify all traffic. LSTM and biLSTM can make full use of the timing information of the sequence to build a classification model, but there are a lot of non-key bytes in the traffic packet, which causes the useful byte information to become scattered. Considering all the timing information will bring redundancy and cannot completely solve the classification problem. CNN utilizes the excellent characteristics of convolution to scan and build models for local important information, thereby avoiding the redundancy problems caused by the dispersion of key information. DeepFinger absorbs the advantages of CNN and achieves accurate clustering of *key-blocks* through deep clustering. Compared with other methods, DeepFinger can also establish multiple detailed device fingerprints for the same IoT device, which is conducive to device identification tasks in a real environment.

5.5. Efficiency analysis

The training efficiency of the Deep Clustering module depends on the training epoch E and the iteration round R . We utilize the

Table 3
Comparative experiment analysis of DeepFinger.

Method	TPR_{AVE}	FTE_{AVE}	FPR_{AVE}
BitCoding	0.7451 ± 0.0031	0.7154 ± 0.0035	0.0416 ± 0.0008
LSTM	0.9301 ± 0.0029	0.9262 ± 0.0036	0.0044 ± 0.0003
biLSTM	0.9448 ± 0.0019	0.9417 ± 0.0201	0.0034 ± 0.0004
CNN	0.9446 ± 0.0013	0.9414 ± 0.0016	0.0035 ± 0.0002
FOM	0.4455 ± 0.0243	0.4123 ± 0.0209	0.0792 ± 0.0035
SOB	0.5006 ± 0.0085	0.4712 ± 0.0086	0.0713 ± 0.0012
MAMPF	0.6280 ± 0.0116	0.5986 ± 0.0118	0.0531 ± 0.0016
Fs-Net	0.9603 ± 0.0023	0.9534 ± 0.0026	0.0072 ± 0.0003
DeepFinger-BIRCH	0.9669 ± 0.0011	0.9417 ± 0.0015	0.0034 ± 0.0003
DeepFinger-DBSCAN	0.9839 ± 0.0011	0.9830 ± 0.0015	0.0010 ± 0.0002
DeepFinger	0.9881 ± 0.0009	0.9874 ± 0.0011	0.0007 ± 0.0001

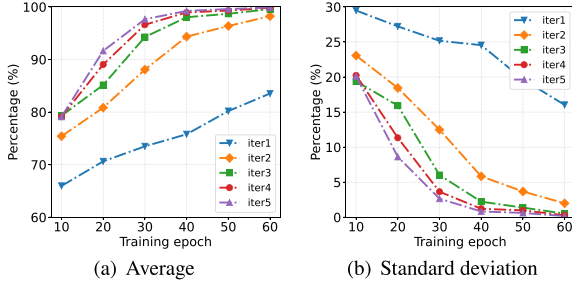


Fig. 9. Metrics of cluster accuracy under different training epochs and rounds.

Table 4
Computational efficiency of each module in DeepFinger.

Module	Time(s)
Data Preprocessing	1.21×10^{-3}
Deep Clustering	1.17×10^3
Keywords Perception	1.09×10^1
Fingerprint Generation	1.89×10^3
Device Identification	4.06×10^{-7}

prediction results accuracy of pseudo-label data to indirectly portray the Deep Clustering module effectiveness. Figure 9(a) and (b) demonstrate the average and standard deviation of the accuracy rate involving two parameters. Obviously, as the training epoch and the iteration round increase, the average accuracy rate of clustering rises while the standard deviation decreases.

It is worthy to note that when the training epoch is relatively small, the accuracy rate approximates to the upper limit instantly at $R = 3$. Increasing the iteration round will not bring the results significantly better, this is because the number of training epoch is too small, resulting in insufficient refinement of the classification model. When $E = 40, R = 5$, the increase in the average accuracy rate gradually stabilizes. Note that the clustering result of some single devices at this time is 100%. We found that when the value of E and R are relatively small, the value of the standard deviation exceeds 0.15, which indicates that the traffic clustering performance of some devices is excellent, but others are relatively poor. As the value of E and R increase, the standard deviation has a clear trend of decreasing and eventually approximates zero, indicating that almost all devices have satisfied accuracy rates.

We evaluate the computational efficiency of each module of DeepFinger and show the result in Table 4. The results of Data Preprocessing and Device Identification are the average time consumption of each flow. Deep Clustering takes approximately 20 min in the training phase since all the parameters have chosen the most complex value. If we adjust the complexity of parameters according to actual needs, the time spent may be reduced by more than half. Keywords Perception takes less than a minute to

Table 5
Running time for identification of each method.

Method	Time(s)
BitCoding	1.21×10^{-3}
LSTM	9.66×10^{-5}
biLSTM	3.48×10^{-4}
CNN	4.55×10^{-5}
DeepFinger	4.06×10^{-7}

find the *stochastic-blocks* and *key-blocks* from all the clusters. Fingerprint Generation and Deep Clustering module spend the same order of time, but this time can be reduced by the multi-process method.

We also compare the time taken by some methods to identify a single flow in the testing phase, and the results are shown in Table 5. DeepFinger possesses the fastest identification speed, the identification time of a single flow is 4.06×10^{-7} s, which achieves a speedup of 100 times than the CNN method.

5.6. Extensibility Analysis

Considering the similarity in design logic between IoT proprietary protocols and IoT public protocols, the method proposed in this paper can theoretically realize IoT device identification not only in the context of proprietary protocol traffic but also in the context of public protocol traffic. We next evaluate the extended performance of Deepfingerprint on public protocol traffic. Considering the similarity in design logic between IoT proprietary protocols and IoT public protocols, the method proposed in this paper can theoretically realize IoT device identification not only in the context of proprietary protocol traffic but also in the context of public protocol traffic. We next evaluate the extended performance of Deepfingerprint on public protocol traffic.

It is worth noting that if most devices use a common public IoT protocol, then the traffic generated by these devices may have similar or identical payloads in the first packet, but discriminative information exists in the data packets later in the same flow. For example, in a message sending flow of the MQTT protocol, the distinguishing application-layer payload is usually in the "Connect Command" packet and the "Publish Message" packet. Therefore, in order to realize the identification of IoT devices using the same public protocol, the data input in DeepFinger needs to be adjusted to a certain extent, and avoid using the first packet of each stream fixedly.

Table 6 shows the performance of Deepfingerprint on three public protocol datasets, and it can be seen that the results on the set overall evaluation metrics are excellent. The Edge-IIoTset dataset contains two public protocol traffic, and our method can fully and effectively realize device identification through the extracted device fingerprints. The IoT Sentinel dataset contains the traffic of

Table 6
Experimental evaluation of the extensibility of DeepFinger.

Dataset	TPR_{AVE}	FTF_{AVE}	FPR_{AVE}
Edge-IoTset	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
IoT Sentinel	0.9929 ± 0.0001	0.9926 ± 0.0001	0.0003 ± 0.0000
MedBioT	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000

various public protocols, and some fingerprints of devices using the HTTP protocol extracted by our method have similarities, resulting in a small number of identification errors in the identification process. The MedBioT dataset only contains one public protocol, our method is also capable of extracting discriminative payload fingerprints of different devices to further achieve accurate device identification. These results fully demonstrate that DeepFinger has a certain extensibility and can be applied to the public protocol traffic, which greatly expands the practical application scenarios of our proposed method.

In addition, by running our method on the public protocol datasets, we can try to use the relevant public protocol format to directly verify the effect of our method on fingerprint extraction. DeepFinger extracts a series of fingerprints for each device and each fingerprint contains key fields of the protocol used by the current IoT device. For a brief and clear comparison, Table 7 shows some representative extracted fingerprints and protocol formats. The extracted fingerprint is displayed in the form of a regular expression of a hexadecimal string, and the protocol format is to disassemble the fingerprint according to the value of each field. The fingerprints extracted by DeepFinger are strongly correlated with the protocol format to a large extent.

For example, for the fingerprints of Fan, DeepFinger successfully identifies the diversity of “Msg Len” field value and “Topic Length” field value and treats them as the *stochastic-blocks* of fixed length, marked with (.?.?). The æTopicg field contains a variable-length *stochastic-block* (*?) and the value of this random block corresponds to the device number by comparing the real message payload. There are 18 fan devices from “fan1” to “fan18” in the dataset. Since the length of the character representing the number is not fixed, it is recognized as a random block of variable length. It should be emphasized again that the design goal of DeepFinger is to identify different types of IoT devices, which means DeepFinger may not distinguish between multiple devices of the same type.

Furthermore, it is worth noting that in the real protocol format, the value of the æTopic Lengthg field is two bytes, but in the fingerprint which we extract, the first byte of these two bytes is a fixed value of “00”, and only the second Bytes are considered *stochastic-block*. The reason for this result is that in our data set, the “Topic Length” field value of all samples is less than 256, and the first byte of the corresponding hexadecimal is “00”. The value at the position of this byte has no instance changes in the processing of DeepFinger, so it will be retained as a fixed value. It also shows that even if our method fails to fully correctly identify the boundary of the real field value of the protocol, the fingerprint proposed by our method is still valid in the final device identification process.

6. Discussion

DeepFinger is based on deep clustering to generate IoT device fingerprints from the packet payloads. We discussed the future improvement direction of DeepFinger from the following four aspects.

Similar payload clustering CNN is selected to construct the Deep Clustering module of DeepFinger as the one-dimensional convolution is more in line with the distribution of key fields and more easily clusters the same *key-blocks*. The purpose of using one-dimensional convolution is to extract the hidden information of

the payload byte sequence for the input of the clustering algorithm. As the kinds of bytes present at each location of the payload are limited, and it is possible for most locations to be fixed for the same protocol, hidden vector Spaces are not particularly complex. In addition, the payload will carry certain time series information, which needs to learn some local pattern from the dimension of the timestep. The characteristics of these IoT protocols led us to choose the simple and efficient one-dimensional convolution. However, more deep learning algorithms and clustering algorithms deserve to be integrated into Deep Clustering in the future. We expect the further extension of DeepFinger to make our method more applicable.

Keyword perception Our *n*-gram-based method perceives keywords from clusters and lacks more discriminative metrics of *key-blocks* and *stochastic-blocks*. In the future, the results of *key-blocks* can be refined by calculating entropy and sorting candidate key blocks through heuristic algorithms.

Fingerprint extensibility In this work, the final goal of DeepFinger is to generate byte-level fingerprints of proprietary-protocol-oriented IoT devices. Although these fingerprints are distinguishable for devices using binary protocols, there are still some ignored key blocks. In future work, we plan to improve the input sequence of DeepFinger. Instead of using the byte sequence of the payload as the original input, we will construct an *n*-bit sequence based on the payload, where *n* is determined according to a specific proprietary protocol. We believe that the *n*-bit sequence will increase the extensibility of our method.

Fingerprint quality evaluation We applied the fingerprints obtained by DeepFinger to IoT device recognition and achieved excellent results. We lack the quality evaluation of the fingerprint itself without prior knowledge of the proprietary protocol format. Nevertheless, we still hope that we can use some similarity calculation methods to evaluate the quality of fingerprints in the future.

7. Related work

How to extract fingerprints to realize IoT device identification has always been the focus of relevant researchers. As a kind of device fingerprint, protocol fingerprint has been studied in many works.

Protocol fingerprint generation The protocol fingerprint generation has been receiving attention from researchers in the past decades. Whether generating byte-level granular fingerprints (Sang et al., 2016; Zhang et al., 2014) or generating bit-level granular fingerprints (Sang and Zhang, 2017; Tao et al., 2016), it is hoped to infer the key fields in the protocol format or the concrete manifestation of such key fields. Delimiter-based method (Cui et al., 2007) splits the payload byte into small blocks to build a bag-of-words model based heuristic delimiters such as whitespaces and printable characters. This method relies heavily on the readable characters of the payload, and lacks applicability in most existing protocols. *N*-gram-based methods (Ye et al., 2009; Zhang et al., 2015) treat packet payload as a subsequence of bytes and use variable length pattern recognition to find distinguishing protocol key fields. However, the strategy ignores the more short *n*-grams ($n < 3$) and leads to filter the true and meaningful protocol key fields. In addition, some protocol reverse engineering work can also obtain protocol fingerprints, but it requires network protocol experts to perform a lot of manual analysis based on protocol specifications document.

IoT device identification In the early period of the development, one would expect that operators directly identify the IoT device through the MAC address and DHCP infrastructure. With the influx of IoT devices, previous methods are no longer applicable, and the traffic generated by the devices has turned into a contemporary research object (Miettinen et al., 2017). The general network traffic

Table 7

Comparison of some obtained fingerprints and public protocol formats

Device	Fingerprints extracted from Deepfinger	Protocol format parsing
Fan	30.?.?00??686f6d652f626564726f6f6d2f66616e.*?2f 6f6e2f7365744f4646	Header Flags:30 Msg Len:?.? Topic Length:00.?.? Topic:home/bedroom/fan.*?/on/set Message:OFF
Fan	30.?.?00.?.?686f6d652f626564726f6f6d2f66616e.*?2f 6f6e2f7365744f4e	Header Flags:30 Msg Len:?.? Topic Length:00.?.? Topic:home/bedroom/fan.*?/on/set Message:ON
Lock	10.?.?00044d5154540402003c00.?.?636c69656e742d30.*?	Header Flags:10 Msg Len:?.? Protocol Name Length:0004 Protocol Name:MQTT Version:04 Connect Flags:02 Keep Alive:60 Client ID Length:00.?.? Client ID: client-0.*? Header Flags:30 Msg Len:?.? Topic Length:00.?.?
Light	30.?.?00.?.?686f6d652f626564726f6f6d2f6c69676874.*?2f 7377697463684f4646	Topic:home/bedroom/light*/switch Message:OFF
Water Level	301f000b57617465725f4c6576656c41444330206c6576656c 20697320.*?0a	Header Flags:30 Msg Len:1f Topic Length:000b Topic:Water_Level Message:ADC0 level is.*? Header Flags:30 Msg Len:27 Topic Length:0018
Temperature and Humidity	3027001854656d70657261747572655f616e645f48756d69 64697479.?.?.?2e.?.?.?20.?.?.?2e.?.?.?20d0a	Topic:Temperature_and_Humidity Message:?.?.?.?.?.?.?.?.?.?.?.?.?.?.?.?.?20d0a Transaction ID:?.?.?.? Protocol ID:0000
Modbus	.?.?.?.?0000002b122b0e0183000003000f70726f636f6e5820 507479204c7464010746542d4d4253560207322e382e322e30	Length:002b Unit ID:12 Function Code:2b MEI type:0e Read Device ID:01 Conformity Level:83 More Follows:00 Next object ID:00 Number of Objects:03 Objects:●●proconX Pty Ltd●●FT-MBSV●●2.8.2.0

model is described by a five-tuple, which depicts the uniqueness of each Internet traffic or IoT traffic. For the IoT devices traffic classification, related work primarily focuses on extracting distinguishable characteristics of the IoT traffic (O'Connor et al., 2019; Sivanathan et al., 2020; Thangavelu et al., 2018). Marchal et al. (2019) proposed AuDI, a system for identifying IoT devices by analyzing device network communications. AuDI analyzes the periodicity and stability of the traffic, then extracts 4 categories and a total of 33 features to model a flow, and finally takes advantage of the kNN clustering algorithm to classify the IoT traffic. Aksoy and Gunes (2019) proposed SysID, a classification system based on device network traffic characteristics and field features, which exploits multiple machine learning classifiers. If the protocol field of the device traffic alters, the corresponding features cannot be proposed, and the device cannot be identified correctly. Msadek et al. (2019) proposed a method to classify IoT devices based on the characteristics of the data packet header. The improved sliding window technology selects the relevant flows. The header field information and flow statistics are applied to classify the traffic by machine learning classifiers. Compared with the model proposed in this paper, these methods all need to manually extract the characteristics and features of the flow or data packet. Besides, several works (Aneja et al., 2018; Lopez-Martin et al., 2017; Ortiz et al., 2019) take advantage of the neural network model to classify traffic and at least require the first 20 data packets of a flow to obtain a classification result, while our method only utilizes the first data packet of a flow.

8. Conclusion

IoT device identification is the foundation of network security management and resource allocation. This paper proposes DeepFinger, an innovative automatic, byte-level, and efficient approach for IoT device fingerprint generation. It is purely based on

the original proprietary protocol traffic and does not need the pertinent protocol structural information or manual search for payload features. DeepFinger combines the classification model established by CNN with the clustering model established by K-Means and realizes the accurate clustering of traffic through the iterative training of pseudo-labels and clusters. In the real traffic payload dataset, DeepFinger can uniquely identify IoT devices with effectiveness. The identification time of a single flow is controlled at the nanometer level, which is two orders of magnitude faster than CNN. In addition, the extensibility analysis shows that DeepFinger not only targets proprietary protocol traffic but also effectively solves the fingerprint extraction of public-protocol-oriented IoT devices.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Yafei Sang: Conceptualization, Data curation, Formal analysis, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Jisong Yang:** Conceptualization, Formal analysis, Investigation, Methodology, Resources, Writing – review & editing. **Yongzheng Zhang:** Data curation, Funding acquisition, Project administration, Resources, Supervision. **Shuhao Li:** Resources, Funding acquisition, Project administration, Supervision. **Peng Chang:** Data curation, Resources, Supervision.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Key Research and Development Program of China (Grant no. 2018YFB0804702), and the National Natural Science Foundation of China (Grant no. U1736218)

References

- Aksoy, A., Gunes, M.H., 2019. Automated IoT device identification using network traffic. In: ICC 2019–2019 IEEE International Conference on Communications (ICC). IEEE, pp. 1–7.
- Alsallakh, B., Kokhlikyan, N., Miglani, V., Yuan, J., Reblitz-Richardson, O., 2020. Mind the PAD–CNNs can develop blind spots. arXiv preprint arXiv:2010.02178.
- Aneja, S., Aneja, N., Islam, M.S., 2018. IoT device fingerprint using deep learning. In: 2018 IEEE International Conference on Internet of Things and Intelligence System (IOTAIS). IEEE, pp. 174–179.
- Apthorpe, N., Reisman, D., Feamster, N., 2017. A smart home is no castle: privacy vulnerabilities of encrypted IoT traffic. arXiv preprint arXiv:1705.06805.
- Beddoe, M.A., 2004. Network protocol analysis using bioinformatics algorithms. *Toorcon* 26 (6), 1095–1098.
- Cui, W., Kannan, J., Wang, H.J., 2007. Discoverer: automatic protocol reverse engineering from network traces. In: USENIX Security Symposium, pp. 1–14.
- Ferrag, M. A., Friha, O., Hamouda, D., Maglaras, L., Janicke, H., 2022. Edge-IIoTset: a new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning.
- Guerra-Manzanares, A., Medina-Galindo, J., Bahsi, H., Nömm, S., 2020. Medbiot: generation of an IoT botnet dataset in a medium-sized IoT network. In: ICISSE, pp. 207–218.
- Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I., Salakhutdinov, R. R., 2012. Improving neural networks by preventing co-adaptation of feature detectors. arXiv preprint arXiv:1207.0580.
- Hubballi, N., Swarnkar, M., 2018. *bitcoding*: network traffic classification through encoded bit level signatures. *IEEE/ACM Trans. Netw.* 26 (5), 2334–2346.
- Jafari, H., Omotere, O., Adesina, D., Wu, H.-H., Qian, L., 2018. IoT devices fingerprinting using deep learning. In: MILCOM 2018–2018 IEEE Military Communications Conference (MILCOM). IEEE, pp. 1–9.
- Kingma, D. P., Ba, J., 2014. Adam: a method for stochastic optimization. arXiv preprint arXiv:1412.6980.
- Korczyński, M., Duda, A., 2014. Markov chain fingerprinting to classify encrypted traffic. In: IEEE INFOCOM 2014–IEEE Conference on Computer Communications. IEEE, pp. 781–789.
- Lanze, F., Panchenko, A., Braatz, B., Zinnen, A., 2012. Clock skew based remote device fingerprinting demystified. In: 2012 IEEE Global Communications Conference (GLOBECOM). IEEE, pp. 813–819.
- Liu, C., Cao, Z., Xiong, G., Gou, G., Yiu, S.-M., He, L., 2018. Mampf: encrypted traffic classification based on multi-attribute Markov probability fingerprints. In: 2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS). IEEE, pp. 1–10.
- Liu, C., He, L., Xiong, G., Cao, Z., Li, Z., 2019. Fs-net: a flow sequence network for encrypted traffic classification. In: IEEE INFOCOM 2019–IEEE Conference on Computer Communications. IEEE, pp. 1171–1179.
- Lopez-Martin, M., Carro, B., Sanchez-Esguevilas, A., Lloret, J., 2017. Network traffic classifier with convolutional and recurrent neural networks for internet of things. *IEEE Access* 5, 18042–18050.
- Maiti, R.R., Siby, S., Sridharan, R., Tippenhauer, N.O., 2017. Link-layer device type classification on encrypted wireless traffic with cots radios. In: European Symposium on Research in Computer Security. Springer, pp. 247–264.
- Marchal, S., Miettinen, M., Nguyen, T.D., Sadeghi, A.-R., Asokan, N., 2019. Audi: toward autonomous IoT device-type identification using periodic communication. *IEEE J. Sel. Areas Commun.* 37 (6), 1402–1412.
- Meidan, Y., Bohadana, M., Shabtai, A., Ochoa, M., Tippenhauer, N. O., Guarnizo, J. D., Elovici, Y., 2017. Detection of unauthorized IoT devices using machine learning techniques. arXiv preprint arXiv:1709.04647.
- Miettinen, M., Marchal, S., Hafeez, I., Asokan, N., Sadeghi, A.-R., Tarkoma, S., 2017. IoT sentinel: automated device-type identification for security enforcement in IoT. In: 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS). IEEE, pp. 2177–2184.
- Msadek, N., Soua, R., Engel, T., 2019. IoT device fingerprinting: machine learning based encrypted traffic analysis. In: 2019 IEEE Wireless Communications and Networking Conference (WCNC). IEEE, pp. 1–8.
- Needleman, S.B., Wunsch, C.D., 1970. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *J. Mol. Biol.* 48 (3), 443–453.
- O'Connor, T., Mohamed, R., Miettinen, M., Enck, W., Reaves, B., Sadeghi, A.-R., 2019. Homesnitch: behavior transparency and control for smart home IoT devices. In: Proceedings of the 12th Conference on Security and Privacy in Wireless and Mobile Networks, pp. 128–138.
- Ortiz, J., Crawford, C., Le, F., 2019. Devicemien: network device behavior modeling for identifying unknown IoT devices. In: Proceedings of the International Conference on Internet of Things Design and Implementation, pp. 106–117.
- Oser, P., Kargl, F., Lüders, S., 2018. Identifying devices of the internet of things using machine learning on clock characteristics. In: International Conference on Security, Privacy and Anonymity in Computation, Communication and Storage. Springer, pp. 417–427.
- Riad, K., Huang, T., Ke, L., 2020. A dynamic and hierarchical access control for IoT in multi-authority cloud storage. *J. Netw. Comput. Appl.* 160, 102633.
- Riyaz, S., Sankhe, K., Ioannidis, S., Chowdhury, K., 2018. Deep learning convolutional neural networks for radio identification. *IEEE Commun. Mag.* 56 (9), 146–152.
- Sang, Y., Li, S., Zhang, Y., Xu, T., 2016. Prodigger: towards robust automatic network protocol fingerprint learning via byte embedding. In: 2016 IEEE Trustcom/BigDataSE/ISPA. IEEE, pp. 284–291.
- Sang, Y., Zhang, Y., 2017. Fingerprinting protocol at bit-level granularity: graph-based approach using cell embedding. In: 2017 IEEE 23rd International Conference on Parallel and Distributed Systems (ICPADS). IEEE, pp. 266–275.
- Shen, M., Wei, M., Zhu, L., Wang, M., 2017. Classification of encrypted traffic with second-order Markov chains and application attribute bigrams. *IEEE Trans. Inf. Forensics Secur.* 12 (8), 1830–1843.
- Sivanathan, A., Gharakheili, H.H., Loi, F., Radford, A., Wijenayake, C., Vishwanath, A., Sivaraman, V., 2018. Classifying IoT devices in smart environments using network traffic characteristics. *IEEE Trans. Mob. Comput.* 18 (8), 1745–1759.
- Sivanathan, A., Gharakheili, H.H., Sivaraman, V., 2020. Detecting behavioral change of IoT devices using clustering-based network traffic modeling. *IEEE Internet Things J.* 7 (8), 7295–7309.
- Tao, S., Yu, H., Li, Q., 2016. Bit-oriented format extraction approach for automatic binary protocol reverse engineering. *IET Commun.* 10 (6), 709–716.
- Thangavelu, V., Divakaran, D.M., Sairam, R., Bhunia, S.S., Gurusamy, M., 2018. Deft: a distributed IoT fingerprinting technique. *IEEE Internet Things J.* 6 (1), 940–952.
- Ye, M., Xu, K., Wu, J., Po, H., 2009. Autosig-automatically generating signatures for applications. In: 2009 Ninth IEEE International Conference on Computer and Information Technology, vol. 2. IEEE, pp. 104–109.
- Zhang, Y., Xu, T., Wang, Y., Sun, J., Zhang, X., 2015. A Markov random field approach to automated protocol signature inference. In: International Conference on Security and Privacy in Communication Systems. Springer, pp. 459–476.
- Zhang, Z., Zhang, Z., Lee, P.P., Liu, Y., Xie, G., 2014. Proword: an unsupervised approach to protocol feature word extraction. In: IEEE INFOCOM 2014–IEEE Conference on Computer Communications. IEEE, pp. 1393–1401.
- Zscaler, 2020. What things are in your network? Website <https://www.zscaler.com/resources/infographics/loT-report-2020.pdf>.

Sang Yafei received B.Eng. and Ph.D. In 2018, he received his Ph.D. from the Institute of Computing Technology, Chinese Academy of Sciences. His research interests include network and information security.

Jisong Yang received the B.Eng. from Tianjin University, China, in 2019. He is currently a graduate student studying for M.Eng. in the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network security and artificial intelligence.

Yongzheng Zhang received the B.Eng. and Ph.D. degrees from the Harbin Institute of Technology, China, in 2001 and 2006, respectively. He is currently a Full Professor and Ph.D. Supervisor with the Institute of Information Engineering, Chinese Academy of Sciences, China. He was honored with the first prize of the Chinese National Award for Science and Technology Progress in 2011. His research interests include network and information security, in particular, cyberspace security situational awareness. He has authored over 60 papers in refereed journals and conference proceedings.

Shuhao Li received the B.Eng. and M.Eng. degrees from the Harbin Institute of Technology, China, in 2006 and 2008, respectively, and the Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, China, in 2012. He is currently an Associate Professor with the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network and information security, in particular, intrusion detection and defense.

Peng Chang received the B.Eng. and M.Eng. degrees from the Harbin Institute of Technology, China, in 2008 and 2010, respectively. He is currently an Associate Professor with the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network and information security, in particular, mobile malware attack detection. He was honored with the second prize of Science and Technology of China Communication Society in 2016.